InsureDesk

Commercial Insurance Management System

Software Requirements Specification

1. Introduction

1.1 Purpose

This document is the Software Requirements Specification (SRS) for InsureDesk, a full-stack commercial insurance policy management system. It describes the functional and non-functional requirements of the system in a format intended for developers, testers, and project stakeholders. This document follows the IEEE 830 standard for SRS documentation and serves as the primary agreement on what the system must do before any design decisions are finalized.

1.2 Scope

InsureDesk is a web-based application that digitizes and automates the complete lifecycle of commercial insurance policies. It replaces fragmented, paper-based workflows with an integrated platform that connects four types of users: customers, agents, claims officers, and administrators. The system covers the following functional areas: user registration and authentication, insurance plan browsing, policy request and underwriting, payment processing, claims filing and adjudication, policy renewal, real-time notifications, and administrative oversight with analytics. The system does not handle external payment gateway integration, regulatory compliance reporting, or email delivery in the current version.

1.3 Definitions and Abbreviations

The following terms and abbreviations are used throughout this document.

- Policy — A formal insurance contract between InsureDesk and a customer, created after a quote is approved and payment is made.
- Claim — A formal request by a customer for financial compensation under an active policy.
- Plan — A specific coverage tier (such as Basic, Standard, or Premium) offered under an insurance type.
- Base Premium — The fixed starting price of a plan before risk-based adjustments are applied.

- Insurance Type — A category of insurance, such as General Liability or Auto Insurance.
- JWT — JSON Web Token. A signed, stateless token used to authenticate API requests.
- SignalR — An ASP.NET Core library that provides real-time, bidirectional communication between the server and connected browser clients using WebSockets.
- EF Core — Entity Framework Core, the ORM used to manage all database interactions.
- RBAC — Role-Based Access Control. A security model where permissions are assigned to roles, not individuals.
- DTO — Data Transfer Object. A plain object used to carry data between the API and its clients.
- SRS — Software Requirements Specification.
- API — Application Programming Interface.

1.4 Overview

The remainder of this document is organized as follows. Section 2 provides an overall description of the product, including its context, user roles, operating environment, and constraints. Section 3 details the functional requirements organized by feature area, including user stories for each module. Section 4 covers external interface requirements. Section 5 describes non-functional requirements. Section 6 specifies business rules and data constraints. Section 7 describes the key workflows of the system.

2. Overall Description

2.1 Product Perspective

InsureDesk is a standalone, three-tier web application. It is not a replacement for or an extension of any existing system. The architecture consists of an Angular 19 single-page application served in the browser, an ASP.NET Core 8 REST API that handles all business logic and data access, and a Microsoft SQL Server relational database managed through Entity Framework Core. The frontend communicates with the backend exclusively through HTTP and JSON. Real-time event delivery is handled through a persistent SignalR WebSocket connection established when a user logs in. All business logic, data validation, and authorization are enforced server-side. The frontend is responsible only for rendering, navigation, and user interaction.

2.2 Product Functions

At a high level, InsureDesk provides the following capabilities. Users can register, log in, and reset passwords using JWT-based authentication. Customers can browse insurance types and plans, submit policy requests, calculate premiums, make payments, file and track claims, and renew expiring policies. Agents review and approve or reject policy requests, activate policies after payment, and monitor their assigned customer portfolio. Claims officers view, adjudicate, and close claims while maintaining an immutable audit trail. Administrators manage all users, monitor system-wide analytics, and have full read access to all records in the system. The notification system delivers real-time alerts via SignalR to the appropriate users whenever domain events occur.

2.3 User Roles

InsureDesk supports four user roles. Each role has a distinct set of responsibilities and access privileges.

The Admin role is seeded automatically on first application startup with the credentials admin@insurance.com and the password Admin@123. Admins manage all user accounts, have unrestricted read access to all records, and view system-wide analytics. They create agent and claims officer accounts.

The Agent role is created by an admin. Agents are responsible for reviewing incoming policy requests from customers, approving or rejecting them with documented reasons, activating policies after confirming payment, and monitoring their assigned customer portfolio through a dedicated dashboard.

The Customer role is created through self-registration. Customers browse insurance plans, request policies, make payments, file claims against active policies, and track all their activity through a personal dashboard.

The Claims Officer role is created by an admin. Claims officers review all submitted claims, examine uploaded documents, transition claims through the adjudication workflow, and record settlement amounts. Every status change they make is logged in an immutable audit trail.

2.4 Operating Environment

The backend API is built on ASP.NET Core 8 and runs on .NET 8. It requires Microsoft SQL Server 2019 or later as its database, though SQL Server LocalDB is acceptable in development environments. The API listens on http://localhost:5202 in development. The Angular 19 frontend requires Node.js 18 or later and Angular CLI 19 or later. It is served at http://localhost:4200 in development. The application targets desktop browsers — Chrome 120 and above, Firefox 120 and above, and Edge 120 and above — as the primary environment. The UI is responsive and supports mobile browsers as a secondary target. WebSocket support in the browser is required for SignalR real-time functionality.

2.5 Design and Implementation Constraints

The backend must implement Clean Architecture with four layers: Domain, Application, Infrastructure, and API. Dependencies flow strictly inward; the Domain layer has no dependencies on any other layer. All database access must go through Entity Framework Core. Direct DDL changes to the database are not permitted; all schema changes must be made through EF Core migrations. Authentication must use JWT Bearer tokens. Session-based or cookie-based authentication is not used for the API. Passwords must be hashed with BCrypt with a minimum cost factor of 10. MD5 and SHA-1 password storage are prohibited. All monetary values are denominated in Indian Rupees and are stored as decimal with two decimal places of precision. The API must expose Swagger documentation accessible at /swagger in development. The frontend must use Tailwind CSS for styling and must support both light and dark display modes.

2.6 Assumptions and Dependencies

The following assumptions apply to this specification. Users access the application through a modern web browser with JavaScript enabled. The SQL Server instance is available and reachable at the connection string defined in appsettings.json. Email delivery for password reset tokens is assumed to function in production; in development,

the reset token is returned directly in the API response. File storage for uploaded documents uses the wwwroot directory of the API server in development environments. The four insurance types and twelve plans are seeded into the database on first application startup. The system depends on the following external libraries and runtimes: .NET 8 SDK, SQL Server 2019+, Entity Framework Core 8, BCrypt.Net, ASP.NET Core SignalR, Swashbuckle for Swagger, Node.js 18+, Angular CLI 19+, Tailwind CSS 3, and the ApexCharts library via its Angular wrapper ng-apexcharts.

3. Functional Requirements

3.1 Authentication and Authorization

3.1.1 Description

This module covers user registration, login, session management via JWT tokens, password reset, and role-based access control. Authentication is entirely stateless on the server; no sessions or cookies are used for the API.

3.1.2 Requirements

FR-AU-01 The system shall allow customers to self-register by providing a first name, last name, email address, and password. Registration shall be available without prior authentication.

FR-AU-02 The system shall reject registration attempts where the email address already exists in the database and shall return an HTTP 400 error with an appropriate message.

FR-AU-03 The system shall hash all user passwords using BCrypt before persisting them. Plaintext passwords shall never be stored.

FR-AU-04 The system shall authenticate users by verifying their email and password. On success, it shall return a signed JWT token containing the user ID, email address, and role.

FR-AU-05 All API endpoints, except public routes under Auth and the insurance catalogue, shall require a valid JWT Bearer token in the Authorization request header.

FR-AU-06 The system shall enforce role-based access control on every protected endpoint using ASP.NET Core authorization policies. Client-side role checks are supplementary only.

FR-AU-07 The system shall allow only Admin users to create Agent and Claims Officer accounts. Customer self-registration shall always assign the Customer role.

FR-AU-08 The system shall support a password reset flow in which a user submits their email address, a secure one-time reset token is generated and returned, and the user sets a new password by submitting the token along with their new password.

FR-AU-09 Password reset tokens shall be invalidated immediately after a single successful use.

FR-AU-10 The frontend shall include an HTTP interceptor that automatically attaches the Authorization: Bearer header to every outgoing API request using the token stored in localStorage.

FR-AU-11 The frontend login form shall include a custom CAPTCHA challenge that must be completed before the form can be submitted.

FR-AU-12 After a successful login, the frontend shall read the role from the JWT and navigate the user to their role-specific dashboard without requiring an additional page load.

3.1.3 User Stories

User Story: *As a customer, I want to register with my name, email, and password so that I can create an account and access the insurance portal.*

User Story: *As a registered user, I want to log in with my email and password so that I receive a token that grants me access to my dashboard.*

User Story: *As a user who has forgotten my password, I want to request a reset link so that I can regain access to my account without contacting support.*

User Story: *As an admin, I want to create accounts for agents and claims officers so that they can log in and perform their responsibilities in the system.*

3.2 Insurance Catalogue

3.2.1 Description

The insurance catalogue is the public-facing product listing of InsureDesk. It presents the available insurance types and their plans and is accessible without requiring a login. On first startup, the system seeds four insurance types with three plans each.

3.2.2 Requirements

FR-IC-01 The system shall maintain four insurance type categories: General Liability, Auto Insurance, Workers Compensation, and Business Interruption.

FR-IC-02 Each insurance type shall contain three plans at Basic, Standard, and Premium tiers, seeded automatically on first application startup.

FR-IC-03 Each plan shall define its name, base premium in rupees, minimum and maximum coverage amounts, and duration in months.

FR-IC-04 The insurance catalogue shall be publicly accessible without authentication, allowing prospective customers to browse plans before registering.

FR-IC-05 Admin users shall be able to create new insurance types and plans through the administration interface.

3.2.3 Seeded Plan Catalogue

The following plans are seeded into the database on first startup.

Insurance Type	Plan	Coverage Range	Base Premium	Duration
General Liability	Basic Liability	Rs. 5L – Rs. 10L	Rs. 8,000	12 months
General Liability	Standard Liability	Rs. 10L – Rs. 20L	Rs. 15,000	24 months
General Liability	Premium Liability	Rs. 20L – Rs. 50L	Rs. 35,000	36 months
Auto Insurance	Basic Auto	Rs. 1L – Rs. 5L	Rs. 5,000	6 months
Auto Insurance	Comprehensive Auto	Rs. 5L – Rs. 10L	Rs. 12,000	12 months
Auto Insurance	Premium Fleet Shield	Rs. 10L – Rs. 30L	Rs. 28,000	24 months
Workers Compensation	Basic Comp	Rs. 1L – Rs. 5L	Rs. 6,000	6 months
Workers Compensation	Standard Support	Rs. 5L – Rs. 15L	Rs. 14,000	12 months
Workers Compensation	Premium Care	Rs. 15L – Rs. 50L	Rs. 32,000	24 months
Business Interruption	Short Term Shield	Rs. 50K – Rs. 2L	Rs. 4,000	6 months
Business Interruption	Standard Continuation	Rs. 2L – Rs. 5L	Rs. 9,000	12 months
Business Interruption	Extended Resilience	Rs. 5L – Rs. 20L	Rs. 25,000	24 months

3.2.4 User Stories

User Story: *As a visitor, I want to browse all available insurance types and plans without logging in so that I can evaluate my options before registering.*

3.3 Business Profile

3.3.1 Description

A Business Profile captures the customer's business details that are used as risk factor inputs in the premium calculation engine. A customer must have a Business Profile before they can request a policy.

3.3.2 Requirements

FR-BP-01 The system shall allow a customer to create a Business Profile containing their business name, industry type, annual revenue, number of employees, and business address.

FR-BP-02 A customer shall have at most one Business Profile. Attempts to create a second profile shall be rejected with an appropriate error.

FR-BP-03 The customer shall be able to update their Business Profile at any time.

FR-BP-04 The system shall redirect a customer attempting to request a policy without a Business Profile to the profile creation page.

FR-BP-05 The Business Profile data shall be supplied automatically to the premium calculation engine when a customer requests a policy.

3.3.3 User Stories

User Story: *As a customer, I want to create a business profile with my company details so that the system can calculate an accurate premium for my insurance request.*

3.4 Policy Request and Premium Calculation

3.4.1 Description

This module allows customers to select an insurance plan, calculate a premium, and submit a policy request. All premium calculations are performed server-side using the customer's business profile data and the selected plan's parameters.

3.4.2 Requirements

FR-PR-01 The system shall allow a logged-in customer to select an insurance type, choose a plan, and specify a desired coverage amount within the plan's defined minimum and maximum bounds.

FR-PR-02 The system shall calculate the policy premium server-side before the request is submitted. Client-submitted premium values shall not be trusted.

FR-PR-03 The premium calculation response shall include a component breakdown showing the contribution of each risk factor to the total premium.

FR-PR-04 On submission, the system shall create a Policy record with the status PendingReview and shall auto-assign an available agent to the policy.

FR-PR-05 The customer shall be able to upload supporting documents when submitting a policy request.

FR-PR-06 The customer shall be able to view all their policies listing the plan name, coverage amount, status, and assigned agent name.

3.4.3 Premium Calculation Formulas

The premium for a policy is computed server-side using the following formulas based on insurance type. BasePremium is the fixed rate defined on the selected plan.

- General Liability: BasePremium + (EmployeeCount × 50) + (IndustryRiskFactor × BasePremium) + (AnnualRevenue ÷ 1,000,000 × 500)
- Auto Insurance: BasePremium + (VehicleCount × 300) + (DriverRiskFactor × BasePremium)
- Workers Compensation: BasePremium + (EmployeeCount × 75) + (HazardFactor × BasePremium) + (AnnualPayroll ÷ 100,000 × 200)
- Business Interruption: BasePremium + (MonthlyRevenue × 0.05) + (BusinessAgeFactor × BasePremium)

All computed amounts are rounded to two decimal places. The system validates that the requested coverage amount falls within the plan's allowed range before performing the calculation.

3.4.4 User Stories

User Story: *As a customer, I want to see a calculated premium based on my business details before committing to a policy so that I can make an informed decision.*

User Story: *As a customer, I want to request a policy by selecting a plan and coverage amount so that my request enters the agent review queue.*

3.5 Agent — Policy Creation

3.5.1 Description

Agents are responsible for reviewing incoming policy requests, making approval or rejection decisions, and activating policies once payment has been confirmed. Each agent can only see policies assigned to them.

3.5.2 Requirements

FR-AG-01 The system shall present an agent with all policy requests that have been assigned to them and currently hold the status PendingReview.

FR-AG-02 The agent shall be able to view the full details of a policy request, including the customer's business profile and the premium calculation breakdown.

FR-AG-03 The agent shall be able to approve a policy, transitioning its status from PendingReview to Approved. A SignalR notification shall be sent to the customer upon approval.

FR-AG-04 The agent shall be able to reject a policy. The rejection reason is mandatory. The policy status shall transition to Rejected and the customer shall be notified.

FR-AG-05 The agent shall be able to activate a policy, transitioning its status from Approved to Active, after confirming that the customer's payment has been received.

FR-AG-06 The system shall calculate and record an agent commission when a policy is processed through an agent. Commission status shall be tracked as Pending until confirmed by the admin.

FR-AG-07 The agent dashboard shall display key performance indicators including the number of pending approvals, active policies, and total assigned customers.

FR-AG-08 The agent shall receive a SignalR notification whenever a new policy request is assigned to them.

3.5.3 User Stories

User Story: *As an agent, I want to review a customer's policy request and business profile so that I can make an informed approval or rejection decision.*

User Story: *As an agent, I want to activate a policy after the customer pays so that their coverage officially begins.*

User Story: *As an agent, I want to see my performance dashboard so that I can track my pending workload and overall activity.*

3.6 Payment Processing

3.6.1 Description

Customers can make payments against approved policies either as a single lump-sum payment or through an installment schedule. Each successful payment generates a uniquely numbered invoice record.

3.6.2 Requirements

FR-PAY-01 The system shall allow a customer to make a payment against a policy that has the status Approved.

FR-PAY-02 The system shall support two payment modes: Lump Sum, in which the full premium is paid in a single transaction, and Installment, in which the premium is divided across a schedule.

FR-PAY-03 Each successful payment shall generate an invoice with a unique number in the format INV-YYYY-NNNNN, where YYYY is the current year and NNNNN is a zero-padded five-digit sequence.

FR-PAY-04 The system shall prevent duplicate payments for the same policy invoice.

FR-PAY-05 The customer shall be able to view their complete payment history, including invoice number, amount paid, payment date, and the associated policy.

FR-PAY-06 The customer shall be able to view the installment payment schedule for a policy, showing each installment's amount and due date.

FR-PAY-07 The system shall send a SignalR notification to the customer confirming each successful payment.

3.6.3 User Stories

User Story: *As a customer, I want to pay my policy premium in full or in instalments so that I can activate my coverage according to my financial preference.*

User Story: *As a customer, I want to receive an invoice after each payment so that I have an official record of the transaction.*

3.7 Claims Management

3.7.1 Description

This module covers the full claims lifecycle from customer submission through officer adjudication and settlement. Every status change is recorded in an immutable audit log to support accountability and traceability.

3.7.2 Requirements

FR-CL-01 The system shall allow a customer to file a claim against an Active policy by providing an incident description, a claimed compensation amount, and one or more supporting documents.

FR-CL-02 The system shall reject claim submissions against policies that are not in Active status and shall return an HTTP 400 error.

FR-CL-03 Each claim shall be assigned a unique claim number in the format CLM-YYYY-NNNNN upon creation and shall be created with the initial status Submitted.

FR-CL-04 The system shall auto-assign an available Claims Officer to the claim upon submission.

FR-CL-05 A Claims Officer shall be able to view all claims in the system, including full claim details, the associated policy and customer information, and all uploaded documents.

FR-CL-06 A Claims Officer shall be able to download or preview documents attached to a claim. Supported formats are PDF, JPG, and PNG.

FR-CL-07 A Claims Officer shall be able to transition a claim through the following status sequence: Submitted to UnderReview, then from UnderReview to either Approved or Rejected, and from Approved to Settled.

FR-CL-08 Remarks from the Claims Officer are mandatory when rejecting a claim. A settlement amount is required when approving a claim.

FR-CL-09 The system shall create an immutable ClaimHistoryLog entry for every claim status transition, recording the new status, timestamp, officer identifier, and remarks. This log cannot be modified or deleted.

FR-CL-10 The customer shall receive a SignalR notification every time their claim status changes.

FR-CL-11 All connected Claims Officers shall receive a SignalR notification broadcast whenever a new claim is submitted anywhere in the system.

3.7.3 User Stories

User Story: *As a customer, I want to file a claim against my active policy with supporting documents so that I can request compensation for an incident.*

User Story: *As a customer, I want to receive real-time updates whenever my claim status changes so that I always know the current state of my case.*

User Story: *As a claims officer, I want to review all submitted claims along with the customer's documents so that I can adjudicate each case with full context.*

User Story: *As a claims officer, I want to approve or reject a claim and record my remarks so that there is a documented reason for every decision.*

3.8 Policy Renewal

3.8.1 Description

Customers can renew expiring or expired policies from their dashboard. Renewal creates a new policy record linked to the original and reintroduces the policy into the standard approval and payment workflow.

3.8.2 Requirements

FR-RN-01 The system shall allow a customer to initiate renewal for a policy that has the status Active or Expired.

FR-RN-02 If the renewal is initiated on or before the policy's expiry date, the new coverage period shall start the day after the current end date, ensuring a gap-free continuation of coverage.

FR-RN-03 If the renewal is initiated after the policy's expiry date, the new coverage period shall begin on the date of renewal. The system shall inform the customer that a gap in coverage exists.

FR-RN-04 Renewal shall create a new Policy record with IsRenewal set to true, the RenewalCount incremented by one, and a PreviousPolicyId referencing the original policy.

FR-RN-05 The renewed policy shall be created with the status PendingReview and shall follow the standard agent review, payment, and activation workflow.

3.8.3 User Stories

User Story: *As a customer, I want to renew my policy before it expires so that my coverage continues without any interruption.*

User Story: *As a customer, I want to be clearly informed if renewing late will create a gap in my coverage so that I can make an informed decision.*

3.9 Administration

3.9.1 Description

The admin module provides complete system oversight. Administrators manage user accounts, monitor system-wide records, and view business analytics through a dedicated dashboard.

3.9.2 Requirements

FR-ADM-01 The system shall automatically seed a default administrator account on first startup with the email admin@insurance.com and the password Admin@123. This password must be changed immediately in production.

FR-ADM-02 An admin shall be able to create an Agent account by providing the agent's full name, email address, password, and area of specialization.

FR-ADM-03 An admin shall be able to create a Claims Officer account by providing the officer's full name and email address.

FR-ADM-04 An admin shall be able to view a list of all users in the system filterable by role, and shall be able to activate or deactivate any user account. Deactivated accounts cannot log in.

FR-ADM-05 An admin shall have read access to all policies, claims, and payments across all customers in the system.

FR-ADM-06 The admin dashboard shall display system-wide metrics including total registered users, total active policies, total premium revenue collected, number of open claims, and the claim resolution rate.

FR-ADM-07 The admin analytics page shall render interactive charts showing premium revenue over time, policy volume by insurance type, and claims volume over time, powered by ApexCharts.

3.9.3 User Stories

User Story: *As an admin, I want to create accounts for agents and claims officers so that they can be onboarded and begin working in the system.*

User Story: *As an admin, I want to deactivate a user account when necessary so that I can revoke system access without permanently deleting data.*

User Story: *As an admin, I want to view system-wide analytics so that I can monitor the health and performance of the business.*

3.10 Real-Time Notifications

3.10.1 Description

InsureDesk uses ASP.NET Core SignalR to push real-time notifications to connected users. Notifications are persisted in the database before being pushed, ensuring they are available to users who were offline when the event occurred.

3.10.2 Requirements

FR-NT-01 The system shall maintain a SignalR hub at the endpoint /notificationHub. The hub shall require authentication; unauthenticated connection attempts shall be rejected.

FR-NT-02 Each authenticated user shall be placed into a SignalR group identified by their user ID upon connection, ensuring notifications are delivered only to the intended recipient.

FR-NT-03 Every notification shall be persisted as a Notification record in the database before being pushed to the connected client, so that users who are offline can retrieve their notifications upon reconnecting.

FR-NT-04 The system shall deliver notifications for the following events: policy approved by agent, policy rejected by agent, policy activated, new policy assigned to agent, claim status changed, new claim submitted, payment recorded, and policy renewal initiated.

FR-NT-05 A customer shall be able to view their full notification inbox showing all received notifications with read and unread status.

FR-NT-06 A customer shall be able to mark individual notifications as read.

3.10.3 User Stories

User Story: *As a customer, I want to receive instant in-app alerts when my policy or claim status changes so that I can take action without having to refresh the page.*

User Story: *As an agent, I want to be notified immediately when a new policy request is assigned to me so that I can begin the review without delay.*

User Story: *As a claims officer, I want to be notified when a new claim is submitted so that I can begin the adjudication process promptly.*

4. External Interface Requirements

4.1 User Interface

The user interface is an Angular 20 single-page application styled with Tailwind CSS. The application supports both light and dark display modes, toggleable by the user with the preference persisted across sessions. All pages are responsive and optimized for desktop browsers at 1280 pixels wide and above, with mobile support as a secondary target. Form validation errors are displayed inline adjacent to the relevant field. Navigation menus are role-aware and show only items relevant to the currently logged-in user's role. Interactive analytics charts on the admin dashboard are rendered using ApexCharts via the ng-apexcharts Angular wrapper. Loading states are communicated to the user through visible indicators while data is being fetched.

4.2 API Interface

The backend exposes a RESTful HTTP API with the base path /api. All responses use JSON as the content type. The API server runs on http://localhost:5202 in development environments, and Swagger documentation is accessible at http://localhost:5202/swagger. Every protected endpoint requires the Authorization: Bearer header containing a valid JWT. Validation errors return HTTP 400 with a structured error body. Authentication failures return HTTP 401. Authorization failures return HTTP 403. All unhandled exceptions are caught by a global exception handler that returns a structured JSON error response without exposing stack traces to the client. The API enables CORS for the Angular frontend origin.

The API exposes twelve controllers. AuthController handles registration, login, and password reset. PolicyController manages the full policy lifecycle including premium calculation, approval, rejection, activation, and renewal. ClaimController handles claim filing, status updates, and history retrieval. PaymentController manages payments, schedules, and invoice generation. AdminController provides user management and dashboard statistics. BusinessProfileController manages customer business profiles. InsuranceTypeController and PlanController manage the product catalogue. NotificationsController retrieves and updates notification records. AnalyticsController serves chart data. AgentDashboardController provides agent-specific KPIs. SupportInquiryController handles contact form submissions.

4.3 Communication Interface

Frontend-to-API communication uses HTTPS in production environments, with HTTP acceptable only for local development. The SignalR hub uses WebSocket transport as the primary protocol and falls back to Server-Sent Events and Long Polling when

WebSocket is unavailable. CORS policy explicitly whitelists allowed origins; wildcard CORS is not permitted in production.

5. Non-Functional Requirements

5.1 Performance

Standard API responses for list and detail endpoints shall complete within 500 milliseconds under normal operating conditions. The premium calculation endpoint shall return a result within 200 milliseconds. SignalR notifications shall be delivered to connected clients within two seconds of the triggering event. The Angular SPA shall complete its initial load within three seconds on a standard broadband connection.

5.2 Security

All passwords shall be stored as BCrypt hashes with a minimum cost factor of ten. JWT tokens shall be signed with a secret key of at least 256 bits; the key shall not be committed to source control. JWT tokens shall have a configurable expiry time and shall be rejected once expired. All request DTOs shall be validated server-side; unvalidated inputs shall never reach database queries. File uploads shall be validated for file type and size before storage to prevent arbitrary file execution. HTTPS shall be enforced in production, with HTTP redirecting to HTTPS. The default admin password shall be changed immediately after first production deployment.

5.3 Reliability

EF Core migrations shall be applied automatically on every application startup to ensure the database schema is always in sync with the deployed code version. All claim status transitions and their corresponding ClaimHistoryLog entries shall be committed atomically within a single database transaction; partial updates are not permitted. Notification records shall be persisted to the database before being pushed via SignalR, so that no notification is lost if the push fails. Payment records and invoices shall be created within a database transaction to prevent orphaned invoice records.

5.4 Usability

The insurance catalogue and plan browsing experience shall be fully usable without an account, allowing prospective customers to evaluate options before registering. The application shall provide clear success and error feedback for every user action. Navigation shall be role-aware so that users are not presented with routes or actions that do not apply to their role. The application shall support dark mode to accommodate user preferences and reduce eye strain in low-light environments.

5.5 Maintainability

Business logic shall reside exclusively in the Application layer. Controllers shall not contain business logic. All service classes shall implement interfaces defined in the Application layer; controllers shall depend on interfaces rather than concrete implementations. All database schema changes shall be made through EF Core migrations. Angular services shall encapsulate all HTTP communication; components shall not call HttpClient directly. The solution shall include unit tests in Application.Tests and Domain.Tests, and integration tests in API.Tests and Infrastructure.Tests.

6. Business Rules

6.1 Access Control

A customer may only view, modify, or act upon their own policies, claims, and payments. Accessing another customer's records shall result in an HTTP 403 or HTTP 404 response. An agent may only view policies and customers that are assigned to them. A claims officer has read access to all policies and customer information necessary for claim adjudication but cannot modify policy records. An admin has unrestricted read and write access to all entities in the system. Only admin users may create agent and claims officer accounts; customer self-registration always assigns the Customer role.

6.2 Policy Rules

A policy may only be created for a coverage amount that falls within the plan's defined minimum and maximum bounds. The policy status progresses in the following fixed sequence: PendingReview to Approved to Active. Status transitions cannot be skipped. A policy may only be rejected while it holds PendingReview status, and a rejection reason is mandatory. A policy transitions to Active only after the agent confirms that payment has been received. Claims may only be filed against policies with the status Active.

6.3 Claim Rules

The permitted claim status sequence is: Submitted, then UnderReview, then either Approved or Rejected, with Approved leading to Settled as the terminal state. Rejected is also a terminal state. A claim cannot be re-opened after reaching Settled or Rejected. A ClaimHistoryLog entry must be created before each status update is committed. Settlement amount is required when approving a claim. Remarks are required when rejecting a claim.

6.4 Payment Rules

Payments may only be made against policies with the status Approved. Duplicate payments for the same invoice are rejected. Each successful payment creates exactly one invoice with a unique invoice number. All monetary values are stored as decimal with two decimal places of precision and are denominated in Indian Rupees. No currency conversion is performed.

6.5 Data Integrity

Soft deletes are used throughout the system. No domain records are permanently removed from the database. A customer may have at most one Business Profile. The user email address must be unique across all roles; duplicate email registration is

rejected. Unique database indexes are enforced on User.Email, Policy.PolicyNumber, Claim.ClaimNumber, and Payment.InvoiceNumber.

6.6 Identifier Formats

Policy numbers follow the format CIPMS-[TYPE]-YYYY-NNNNN, where TYPE is a two-letter code for the insurance type (for example GL for General Liability), YYYY is the four-digit year, and NNNNN is a five-digit zero-padded sequence per type per year. An example is CIPMS-GL-2026-00001. Claim numbers follow the format CLM-YYYY-NNNNN, for example CLM-2026-00001. Invoice numbers follow the format INV-YYYY-NNNNN, for example INV-2026-00001.

7. Key Workflows

7.1 Policy Purchase Workflow

A customer begins by browsing the public insurance catalogue and selecting an insurance type and a specific plan. If the customer does not yet have a Business Profile, the system redirects them to create one before proceeding. The customer then specifies a coverage amount within the plan's bounds. The system calculates the premium server-side and returns a detailed cost breakdown. The customer reviews the premium and submits the policy request, optionally attaching supporting documents. The system creates a Policy record with status PendingReview and automatically assigns it to an available agent. The agent receives a real-time notification. The agent reviews the request, examining the business profile and premium details, and either approves or rejects the policy with a documented reason. Upon approval, the customer is notified and proceeds to the payment screen, where they can choose a lump-sum payment or an installment schedule. Once the payment is made and an invoice is generated, the agent confirms receipt and activates the policy. The status transitions to Active and the customer receives a confirmation notification. Coverage begins immediately.

7.2 Claim Processing Workflow

A customer files a claim against one of their Active policies by providing an incident description, a claimed compensation amount, and one or more supporting documents. The system creates a Claim record with the status Submitted and auto-assigns a Claims Officer. All connected Claims Officers receive a real-time notification of the new submission. The assigned officer opens the claim, reviews the customer's details, the underlying policy, and all attached documents. The officer transitions the claim to UnderReview while conducting the assessment. The officer then either approves the claim by recording a settlement amount, or rejects it with mandatory remarks. An

immutable ClaimHistoryLog entry is created for every status transition. If approved, the officer subsequently marks the claim as Settled once the settlement payment has been processed. The customer receives a SignalR notification at every stage of the process.

7.3 Policy Renewal Workflow

A customer identifies an expiring or already-expired policy on their dashboard and initiates a renewal. If the renewal is requested on or before the policy's expiry date, the system creates a new policy with a start date equal to the day after the current end date, ensuring there is no gap in coverage. If the renewal is requested after the expiry date, the system creates the new policy with a start date equal to the renewal date and displays a notification informing the customer that a coverage gap exists. In both cases, the new policy is linked to the original via a PreviousPolicyId reference, with IsRenewal set to true and RenewalCount incremented. The renewed policy begins with status PendingReview and follows the same agent review, payment, and activation workflow as a new policy purchase.

8. Calculations and Computation Logic

This section documents every calculation performed by the InsureDesk system. There are five core computation engines: the risk-based premium engine, the loyalty discount engine, the agent commission engine, the analytics and net revenue engine, and the remaining coverage engine. All calculations are executed server-side within Application layer service classes. No client-submitted computed value is trusted or persisted without server-side recomputation.

8.1 Risk-Based Premium Engine

8.1.1 Overview

The premium engine is invoked whenever a customer requests a quote. It derives a risk-adjusted premium using a Base Rate anchored to the selected coverage amount, then multiplies that base by three independent risk factors drawn from the customer's Business Profile: Industry Risk, Employee Count, and Annual Revenue. The final premium is the sum of the base rate and all three risk components, rounded to two decimal places.

8.1.2 Base Rate

The Base Rate is the starting monetary value from which risk adjustments are made. It is computed from the customer's chosen coverage amount and is capped so that it never exceeds two percent of that coverage amount. This cap prevents the base rate from dominating the premium on very large coverage policies.

$$\text{BaseRate} = \text{MIN}(\text{Plan.BasePremium},\ \text{SelectedCoverageAmount} \times 0.02)$$

Variable	Source	Description
Plan.BasePremium	Plan entity, seeded per plan tier	The plan's fixed starting premium (e.g., Rs. 8,000 for Basic GL, Rs. 35,000 for Premium GL)
SelectedCoverageAmount	Customer input, validated by system	The coverage amount chosen by the customer, constrained between Plan.MinCoverageAmount and Plan.MaxCoverageAmount
0.02	System constant	The 2% cap rate applied to coverage amount to constrain the base rate

If the plan's BasePremium is already below 2% of the selected coverage, it is used as-is. The cap only activates when BasePremium exceeds 2% of coverage, which can occur on low-coverage, high-tier plans.

8.1.3 Industry Risk Multiplier

The Industry Risk multiplier reflects the statistical loss exposure of the customer's industry category. It is expressed as a decimal fraction and is applied directly to the BaseRate, producing an additive industry risk charge.

IndustryRiskCharge = BaseRate × IndustryRiskFactor

The IndustryRiskFactor is looked up from a predefined industry classification table keyed on the Industry field of the customer's BusinessProfile. Higher-hazard sectors such as construction, mining, and heavy manufacturing carry higher factors. Lower-risk sectors such as professional services and retail carry lower factors. The factor range is 0.05 (minimum, lowest-risk industry) to 0.50 (maximum, highest-risk industry).

8.1.4 Employee Count Multiplier

The Employee Count multiplier scales a per-employee risk charge by the size of the customer's workforce. A larger workforce increases aggregate liability exposure across General Liability, Workers Compensation, and Business Interruption policy types. The per-employee charge is applied uniformly regardless of plan tier.

EmployeeRiskCharge = BusinessProfile.EmployeeCount × PerEmployeeRate

PerEmployeeRate is a system constant defined in the Application layer. Its value is Rs. 50 per employee for General Liability and Business Interruption plans, and Rs. 75 per employee for Workers Compensation plans, reflecting the greater per-person claim exposure of that insurance type.

8.1.5 Annual Revenue Multiplier

The Annual Revenue multiplier captures the financial scale risk of the insured business. A business with higher revenue has greater operational exposure and potential claim magnitude. The charge is computed as a fraction of the business's annual revenue.

RevenueRiskCharge = BusinessProfile.AnnualRevenue × RevenueRiskRate

RevenueRiskRate is a system constant set at 0.00005 (0.005% of annual revenue). This ensures that the revenue component contributes proportionally without overshadowing the base rate for mid-size businesses.

8.1.6 Final Premium Formula

The total computed premium is the sum of the Base Rate and all three risk charges. This amount is stored on Policy.PremiumAmount upon policy creation and is never recalculated after that point.

PremiumAmount = BaseRate + IndustryRiskCharge + EmployeeRiskCharge + RevenueRiskCharge

Expanding all terms in full:

PremiumAmount = MIN(Plan.BasePremium, Coverage × 0.02) + (BaseRate × IndustryRiskFactor) + (EmployeeCount × PerEmployeeRate) + (AnnualRevenue × 0.00005)

The API endpoint POST /api/policy/calculate-premium returns this amount along with a component breakdown object showing each of the four values separately so the customer and the reviewing agent can see exactly how the premium was derived.

8.1.7 Worked Example

Plan: Standard Liability (BasePremium = Rs. 15,000). SelectedCoverageAmount = Rs. 15,00,000. Industry = Manufacturing (IndustryRiskFactor = 0.25). EmployeeCount = 40. AnnualRevenue = Rs. 60,00,000.

- BaseRate: MIN(15,000, 15,00,000 × 0.02) = MIN(15,000, 30,000) = Rs. 15,000
- IndustryRiskCharge: 15,000 × 0.25 = Rs. 3,750
- EmployeeRiskCharge: 40 × 50 = Rs. 2,000
- RevenueRiskCharge: 60,00,000 × 0.00005 = Rs. 300
- PremiumAmount = 15,000 + 3,750 + 2,000 + 300 = Rs. 21,050

8.2 Loyalty Discount — Annual Payment

8.2.1 Overview

InsureDesk rewards customers who pay the full annual premium in a single lump-sum transaction with a five percent loyalty discount. This discount is applied automatically by the payment service at checkout when the customer selects the Annual payment mode. No voucher code or manual intervention is required.

8.2.2 Discount Computation

The discount is computed by subtracting five percent from the total computed premium. The result is the amount the customer is charged when paying annually. The undiscounted premium remains stored on Policy.PremiumAmount for reference and reporting purposes. The Payment.Amount field stores the actual discounted amount collected.

$$\text{AnnualPaymentAmount} = \text{PremiumAmount} \times (1 - 0.05) = \text{PremiumAmount} \times 0.95$$

$$\text{DiscountAmount} = \text{PremiumAmount} \times 0.05$$

Variable	Value / Source	Description
PremiumAmount	Policy.PremiumAmount	The full risk-adjusted premium computed by the premium engine, before any discount
Discount Rate	0.05 (system constant)	Fixed five percent discount applied to all annual lump-sum payments
AnnualPaymentAmount	Computed at payment time	The amount actually charged to the customer and stored on Payment.Amount
DiscountAmount	Computed at payment time	The saving passed to the customer; displayed on the invoice for transparency

8.2.3 Monthly Installment Baseline (No Discount)

When a customer chooses the monthly installment mode, no loyalty discount is applied. The monthly installment amount is simply the full premium divided equally across twelve months. The total paid over twelve months therefore exceeds the annual lump-sum amount by exactly five percent, which is the cost of choosing installments over annual payment.

$\text{MonthlyInstallmentAmount} = \text{PremiumAmount} \div 12$ (rounded up to 2 decimal places)

$$\text{Total paid over 12 months} = \text{MonthlyInstallmentAmount} \times 12 \geq \text{PremiumAmount}$$

$$\text{Annual saving from lump-sum} = (\text{MonthlyInstallmentAmount} \times 12) - \text{AnnualPaymentAmount} = \text{PremiumAmount} \times 0.05$$

8.2.4 Worked Example

Policy premium: Rs. 21,050.

- Annual lump-sum: $21{,}050 \times 0.95 =$ Rs. 19,997.50 (saving of Rs. 1,052.50)

- Monthly installment: 21,050 ÷ 12 = Rs. 1,754.17 per month (total over 12 months: Rs. 21,050.04)
- Difference: Rs. 21,050.04 − Rs. 19,997.50 = Rs. 1,052.54 (approximately 5%)

8.3 Agent Commission Structure

8.3.1 Overview

Every policy that is processed through an agent generates a commission record for that agent. The commission rate is a flat ten percent of the policy's full PremiumAmount, applied uniformly across all plan types, tiers, and insurance categories. The commission is computed once at policy creation time and is not recalculated if the premium is subsequently amended.

8.3.2 Commission Formula

CommissionAmount = Policy.PremiumAmount × 0.10

Variable	Value / Source	Description
PremiumAmount	Policy.PremiumAmount	Full risk-adjusted premium before any loyalty discount is applied
Commission Rate	0.10 (system constant)	Flat ten percent rate applied to all agent-handled policies without exception
CommissionAmount	Stored on Policy record	The rupee value of the agent's commission for this policy

The commission is computed on the undiscounted PremiumAmount, not on the Payment.Amount collected. This means the agent earns commission based on the full risk price of the policy regardless of whether the customer chose the discounted annual payment or monthly installments.

8.3.3 Commission Status Lifecycle

The CommissionStatus field on the Policy record tracks the payment state of the agent's commission. It begins as Pending when the policy is created and transitions to Earned when the agent activates the policy by confirming that the customer's first payment has been received. The transition from Pending to Earned is triggered by the agent's activation action, not by the payment itself being recorded.

CommissionStatus	Trigger	Meaning
Pending	Policy created and assigned to agent	Commission calculated and owed but not yet payable
Earned	Agent activates the policy (Status → Active)	Commission is confirmed and payable to the agent

No commission is generated for policies where AgentId is null, which occurs when a customer completes a self-service policy purchase without agent assistance.

8.3.4 Worked Example

Policy premium: Rs. 21,050. Agent: assigned at policy creation.

- CommissionAmount = 21,050 × 0.10 = Rs. 2,105.00
- CommissionStatus at creation: Pending
- CommissionStatus after agent activation: Earned
- Agent's commission dashboard shows Rs. 2,105.00 under Earned Commissions

8.4 Analytics and Net Revenue Engine

8.4.1 Overview

The admin analytics dashboard computes financial performance metrics in real time from live database records. Two core derived values are Net Revenue and Growth Percentage. These are not stored; they are computed on demand whenever the analytics endpoint is called and returned as part of the AdminDashboardService response.

8.4.2 Gross Premium Revenue

Gross Premium Revenue is the total of all completed payment amounts collected across all policies in the selected period. It counts only Payment records where PaymentStatus is Completed.

GrossPremiumRevenue = SUM(Payment.Amount WHERE Payment.Status = Completed AND Payment.PaymentDate IN period)

8.4.3 Total Settled Claims

Total Settled Claims is the sum of all approved settlement amounts recorded against claims that reached Settled status within the same period. The settlement amount is stored on each Claim record by the Claims Officer at the time of approval.

TotalSettledClaims = SUM(Claim.SettlementAmount WHERE Claim.Status = Settled AND Claim.ResolvedAt IN period)

8.4.4 Net Revenue

Net Revenue represents the financial surplus remaining after claim payouts are subtracted from premium collections. It is the primary profitability indicator on the admin dashboard.

NetRevenue = GrossPremiumRevenue − TotalSettledClaims

NetRevenue can be negative if claim settlements in a period exceed premium collections for that period, which indicates an adverse claims experience. The dashboard renders this value in a distinct colour to flag a deficit.

8.4.5 Growth Percentage

Growth Percentage tracks period-over-period change in net revenue to show whether the business is growing or contracting. It compares the current period's net revenue against the immediately preceding period of identical length.

GrowthPercentage=((NetRevenue_Current−NetRevenue_Previous)÷|NetRevenue_Previous|) × 100

Variable	Description
NetRevenue_Current	Net revenue for the current reporting period (e.g., current calendar month)
NetRevenue_Previous	Net revenue for the immediately prior period of equal length (e.g., previous calendar month)
|NetRevenue_Previous|	Absolute value used as denominator to handle cases where the prior period was a deficit

If NetRevenue_Previous is zero, GrowthPercentage is displayed as N/A rather than causing a division-by-zero error. A positive GrowthPercentage indicates improving profitability; a negative value indicates contraction.

8.4.6 Worked Example

Previous month: Gross = Rs. 4,80,000. Settled claims = Rs. 90,000. NetRevenue_Previous = Rs. 3,90,000.

Current month: Gross = Rs. 5,50,000. Settled claims = Rs. 1,10,000. NetRevenue_Current = Rs. 4,40,000.

- NetRevenue = 5,50,000 − 1,10,000 = Rs. 4,40,000
- GrowthPercentage = ((4,40,000 − 3,90,000) ÷ 3,90,000) × 100 = (50,000 ÷ 3,90,000) × 100 ≈ 12.82%

8.5 Remaining Coverage

8.5.1 Overview

Remaining Coverage is a real-time computed value shown to both the customer and the assigned agent on the policy detail view. It reflects how much of the policy's original coverage limit is still available for future claims, after accounting for all amounts already paid out on Settled claims under that policy.

8.5.2 Formula

Remaining Coverage is not stored in the database. It is computed on demand each time the policy detail is requested, by subtracting the sum of all settled claim amounts from the policy's original SelectedCoverageAmount.

RemainingCoverage=Policy.SelectedCoverageAmount−SUM(Claim.SettlementAmount WHERE Claim.PolicyId = policy.Id AND Claim.Status = Settled)

Variable	Source	Description
SelectedCoverageAmount	Policy.SelectedCoverageAmount	The coverage limit the customer selected when requesting the policy, constrained by the plan's min/max bounds
Claim.SettlementAmount	Set by Claims Officer on approval	The actual rupee amount paid out on a settled claim against this policy
RemainingCoverage	Computed at query time	The live protection still available; displayed as a progress indicator on the policy card

8.5.3 Boundary Conditions

RemainingCoverage is floored at zero. If the sum of settled claim amounts equals or exceeds the SelectedCoverageAmount, RemainingCoverage is shown as Rs. 0.00 rather than a negative value, and the system displays a warning that the policy's coverage limit has been exhausted. New claims can still be submitted while the policy is Active, but the Claims Officer is expected to note the exhausted coverage when making the settlement decision.

RemainingCoverage = MAX (0, SelectedCoverageAmount − SUM(SettlementAmounts))

8.5.4 Worked Example

Policy: SelectedCoverageAmount = Rs. 10,00,000.

- Claim 1 (Settled): SettlementAmount = Rs. 2,50,000
- Claim 2 (Settled): SettlementAmount = Rs. 1,80,000
- Claim 3 (UnderReview): not yet included (only Settled claims count)
- SUM of settled amounts = Rs. 2,50,000 + Rs. 1,80,000 = Rs. 4,30,000
- RemainingCoverage = 10,00,000 − 4,30,000 = Rs. 5,70,000
- Displayed to customer: 'Rs. 5,70,000 of Rs. 10,00,000 remaining (57%)

8.6 Identifier Generation

All primary business identifiers are generated server-side at record creation time. The sequence component is computed by counting existing records of the same type within the current calendar year and incrementing by one. The result is zero-padded to five digits.

Record	Format	Example	Sequence Scope
Policy	CIPMS-[TYPE]-YYYY-NNNNN	CIPMS-GL-2026-00001	Per insurance type code per calendar year
Claim	CLM-YYYY-NNNNN	CLM-2026-00001	Global per calendar year
Invoice	INV-YYYY-NNNNN	INV-2026-00001	Global per calendar year

NNNNN = COUNT(records of same type in current year) + 1, zero-padded to 5 digits

Insurance type codes: GL (General Liability), AU (Auto Insurance), WC (Workers Compensation), BI (Business Interruption). A two-letter type code must be registered for any new insurance type before policies of that type can be issued.